

웹 허니팟 시스템

요구사항 명세서 (SRS)

문서 정보

문서 번호	SRS-HONEYPOT-001
버전	1.0
작성일	2025년
상태	초안 (Draft)

팀 구성

역할	이름	담당 모듈
팀장 / API & 프론트엔드	도현서	api/, dashboard/
팀원 / 백엔드 수집기	원재준	core/ (C 모듈)

본 문서는 웹 허니팟 시스템 개발 프로젝트의 공식 요구사항 명세서입니다.
문서에 기재된 모든 요구사항은 팀원 간 합의를 통해 확정됩니다.

Contents

1 프로젝트 개요	3
1.1 목적	3
1.2 시스템 구성	3
1.3 용어 정의	3
2 팀원별 담당 범위	4
3 기능 요구사항	4
3.1 모듈 1: core/ — C 수집기 (담당: 원재준)	4
3.1.1 FR-C-01: 소켓 서버	4
3.1.2 FR-C-02: HTTP 요청 파서	4
3.1.3 FR-C-03: SQLite 로거	5
3.1.4 FR-C-04: 가짜 HTTP 응답 생성	5
3.1.5 FR-C-05: 공유 헤더 <code>honeypot.h</code>	5
3.2 모듈 2: api/ — Kotlin API 서버 (담당: 도현서)	6
3.2.1 FR-K-01: REST API 엔드포인트	6
3.2.2 FR-K-02: 공격 유형 자동 분류기 (AttackClassifier)	7
3.2.3 FR-K-03: Exposed ORM 사용 규칙	7
3.3 모듈 3: dashboard/ — React 대시보드 (담당: 도현서)	8
3.3.1 FR-R-01: 컴포넌트 구성	8
3.3.2 FR-R-02: 자동 갱신	8
3.3.3 FR-R-03: API 클라이언트	8
4 비기능 요구사항	9
4.1 성능 요구사항	9
4.2 신뢰성 요구사항	9
4.3 유지보수성 요구사항	9
5 인터페이스 명세 (모듈 간 협업)	9
5.1 데이터베이스 스키마	9
5.2 포트 및 경로 규약	10
5.3 INSERT 규약	10
6 Docker 배포 요구사항	10
6.1 컨테이너 구성	11
6.2 Dockerfile 주의사항	11
7 구현 체크리스트	11
7.1 원재준 (core/)	11
7.2 도현서 (api/ + dashboard/)	12
8 통합 및 테스트 계획	12
8.1 통합 순서	12
8.2 단위 테스트 항목	13

9 변경 이력

13

1 프로젝트 개요

1.1 목적

본 시스템은 공격자를 유인하여 실제 공격 트래픽을 수집·분석하는 **웹 허니팟(Web Honeypot)** 시스템이다. 원시 HTTP 요청을 가짜 응답으로 처리하면서 SQLite 데이터베이스에 로그를 기록하고, Kotlin API 서버가 이를 분류하여 React 대시보드로 시각화한다.

1.2 시스템 구성

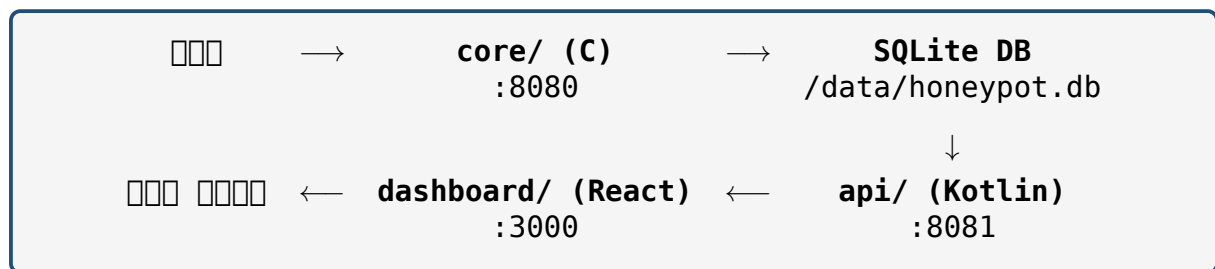


Figure 1: 시스템 아키텍처 개요

1.3 용어 정의

용어	정의
허니팟(Honeypot)	공격자를 유인하기 위해 의도적으로 취약하게 설정된 가짜 서버 시스템
raw socket	운영체제 수준에서 직접 네트워크 패킷을 송수신하는 소켓 인터페이스
attack_type	수집된 요청의 공격 유형 분류 레이블 (SQLi, XSS, 스캔, 브루트포스, 기타)
WAL 모드	SQLite Write-Ahead Logging — 동시 읽기/쓰기 성능을 향상시키는 저널 모드
ISO 8601	날짜·시간 국제 표준 형식 (YYYY-MM-DDTHH:MM:SSZ)
Exposed ORM	Kotlin용 SQL 라이브러리 (JetBrains)
SRS	Software Requirements Specification (소프트웨어 요구사항 명세서)

Table 1: 용어 정의

2 팀원별 담당 범위

담당자	모듈	세부 내용
원재준	core/	C 언어 소켓 서버, HTTP 파서, SQLite 로거, 가짜 응답 생성
도현서	api/	Kotlin/Ktor REST API 서버, 공격 유형 분류기 (AttackClassifier)
도현서	dashboard/	React 대시보드, 차트 시각화, 로그 테이블, 자동 갱신

Table 2: 팀원별 담당 모듈

3 기능 요구사항

3.1 모듈 1: core/ — C 수집기 (담당: 원재준)

3.1.1 FR-C-01: 소켓 서버

요구사항 ID	FR-C-01
명칭	TCP 소켓 서버 구동
설명	raw socket 기반으로 HTTP 요청을 수신하는 서버를 구현한다
입력	네트워크 포트 8080
처리	socket() → SO_REUSEADDR → bind() → listen(backlog=10) → accept() 루프
출력	클라이언트 연결 수락 후 핸들러 스레드 생성
비고	각 연결마다 pthread_create()로 독립 스레드 생성 (동시 연결 처리)

Table 3: FR-C-01 소켓 서버

3.1.2 FR-C-02: HTTP 요청 파서

요구사항 ID	FR-C-02
명칭	HTTP 요청 파싱
설명	raw HTTP 데이터에서 method, path, User-Agent, body를 추출한다
입력	raw HTTP 문자열 (최대 BUF_SIZE = 8192 bytes)
처리	(1) 첫 줄 파싱 → method, path 추출 (2) 헤더 루프 → User-Agent 값 복사 (3) \r\n\r\n 이후 → body 추출
출력	HttpRequest 구조체
예외 처리	body 없는 GET 요청, BUF_SIZE 초과 요청 처리 필수

Table 4: FR-C-02 HTTP 요청 파서

3.1.3 FR-C-03: SQLite 로거

요구사항 ID	FR-C-03
명칭	SQLite 요청 로깅
설명	수신한 HTTP 요청 정보를 SQLite 데이터베이스에 기록한다
초기화	logger_init() 실행 시 DB 파일 및 테이블 자동 생성
삽입 규칙	attack_type 컬럼은 NULL로 삽입 (Kotlin API가 나중에 채움)
성능	WAL 모드 활성화: PRAGMA journal_mode=WAL;
스레드 안전	sqlite3_config(SQLITE_CONFIG_SERIALIZED) 또는 mutex로 동시 접근 보호 필수

Table 5: FR-C-03 SQLite 로거

3.1.4 FR-C-04: 가짜 HTTP 응답 생성

요구사항 ID	FR-C-04
명칭	가짜 HTTP 응답 반환
설명	요청 경로에 따라 공격자가 실제 취약 서버로 인식하도록 가짜 응답을 반환한다

경로 패턴	응답 코드	응답 내용
/admin, /admin/login	200 OK	HTML 관리자 로그인 폼
/.env	200 OK	DB_PASSWORD=supersecret 텍스트
/wp-login.php	200 OK	WordPress 로그인 HTML
/login, /signin	200 OK	일반 로그인 폼 HTML
그 외 모든 경로	404	Not Found

Table 6: FR-C-04 경로별 가짜 응답 매핑

3.1.5 FR-C-05: 공유 헤더 honeypot.h

모든 C 모듈이 공유하는 구조체와 상수를 단일 헤더 파일로 관리한다.

```

1 #ifndef HONEYPOT_H
2 #define HONEYPOT_H
3
4 #define DB_PATH    "/data/honeypot.db"
5 #define PORT       8080
6 #define BUF_SIZE   8192
7
8 typedef struct {
9     char timestamp[32]; /* ISO 8601: "2024-01-01T12:00:00Z" */
10    char ip[46];         /* IPv4 / IPv6 */

```

```

11     char method[8];          /* "GET", "POST" ... */
12     char path[1024];
13     char user_agent[512];
14     char body[4096];
15 } HttpRequest;
16
17 void server_start(int port);
18 void http_parse(const char *raw, HttpRequest *req);
19 void logger_init(const char *db_path);
20 void log_request(const HttpRequest *req);
21 void send_fake_response(int client_fd, const char *path);
22
23 #endif

```

Listing 1: honeypot.h 명세

주의

timestamp 필드는 반드시 ISO 8601 형식(2024-01-01T12:00:00Z)을 사용해야 한다. Kotlin 측 kotlinox.datetime 파서가 이 형식에 맞춰 구현되어 있다.

3.2 모듈 2: api/ — Kotlin API 서버 (담당: 도현서)

3.2.1 FR-K-01: REST API 엔드포인트

엔드포인트	메서드	파라미터	설명
/api/logs	GET	page, limit, type (선택)	로그 목록 페이지네이션 조회
/api/stats	GET	없음	공격 유형별 집계 통계
/api/top-ips	GET	없음	요청 횟수 상위 IP 목록

Table 7: FR-K-01 REST API 엔드포인트 목록

GET /api/logs 응답 형식

```

1 {
2   "total": 1234,
3   "logs": [
4     {
5       "id": 1,
6       "timestamp": "2024-01-01T12:00:00Z",
7       "ip": "1.2.3.4",
8       "method": "POST",
9       "path": "/login",
10      "userAgent": "sqlmap/1.7",
11      "body": "id=1'",
12      "attackType": "SQLi"
13    }
14  ]

```

15 }

GET /api/stats 응답 형식

```
1 {
2   "total": 1234,
3   "byType": {
4     "SQLi": 400,
5     "XSS": 200,
6     "": 150,
7     "": 300,
8     "": 184
9   }
10 }
```

GET /api/top-ips 응답 형식

```
1 [
2   { "ip": "1.2.3.4", "count": 300 },
3   { "ip": "5.6.7.8", "count": 150 }
4 ]
```

3.2.2 FR-K-02: 공격 유형 자동 분류기 (AttackClassifier)

유형	우선순위	판단 기준
SQLi	1	path 또는 body에 select, union, --, ', 1=1 등 SQL 키워드 포함
XSS	2	path 또는 body에 <script, onerror=, javascript:, alert(등 스크립트 패턴 포함
스캔	3	경로가 /.env, /.git, /wp-login.php, /phpmyadmin 등 정찰 대상 패턴에 일치
브루트포스	4	동일 IP가 60초 이내에 로그인 경로(/login, /signin 등)에 10회 이상 요청
기타	5	위 어떤 유형에도 해당하지 않는 요청

Table 8: FR-K-02 공격 유형 분류 규칙

참고

분류기는 attack_type = NULL인 행을 5초 주기 코루틴으로 조회하여 분류 후 UPDATE한다. C 모듈이 INSERT 시 attack_type을 NULL로 남기는 것이 전제 조건이다.

3.2.3 FR-K-03: Exposed ORM 사용 규칙

Exposed 0.52.0 API 호환성을 위해 아래 규칙을 준수한다.

잘못된 사용 (구버전)	올바른 사용 (0.52.0)
Table.select { cond }	Table.selectAll().where { cond }
Table.slice(c1,c2).select { cond }	Table.select(c1,c2).where { cond }
query.limit(n).offset(m)	query.limit(n, offset)

Table 9: FR-K-03 Exposed 0.52.0 API 호환성 규칙

3.3 모듈 3: dashboard/ — React 대시보드 (담당: 도현서)

3.3.1 FR-R-01: 컴포넌트 구성

컴포넌트	기능
StatCard.jsx	공격 유형별 총 건수를 카드 형태로 표시
StatsChart.jsx	공격 유형 분포 파이차트 (recharts 사용)
TopIps.jsx	상위 공격 IP 바차트 (recharts 사용)
LogTable.jsx	유형 필터 + 페이지네이션이 적용된 로그 목록 테이블
App.jsx	전체 레이아웃 조합, 10초 주기 자동 갱신

Table 10: FR-R-01 React 컴포넌트 목록

3.3.2 FR-R-02: 자동 갱신

- App.jsx에서 setInterval을 사용하여 10초마다 세 API를 재호출한다.
- 컴포넌트 언마운트 시 인터벌을 정리(clearInterval)해야 한다.

3.3.3 FR-R-03: API 클라이언트

src/api/index.js에 다음 세 함수를 구현한다.

함수	호출 엔드포인트
fetchLogs(page, limit, type)	GET /api/logs
fetchStats()	GET /api/stats
fetchTopIps()	GET /api/top-ips

Table 11: FR-R-03 API 클라이언트 함수

4 비기능 요구사항

4.1 성능 요구사항

ID	항목	요구 수준
NFR-P-01	동시 연결 처리	수십 개의 동시 TCP 연결을 처리할 수 있어야 한다 (멀티스레드)
NFR-P-02	DB 쓰기 성능	WAL 모드 활성화로 동시 읽기/쓰기 지원
NFR-P-03	대시보드 갱신	10초 주기 자동 갱신, 갱신 실패 시 이전 데이터 유지
NFR-P-04	API 응답 시간	일반 조회 요청에 대해 500ms 이내 응답

Table 12: 성능 요구사항

4.2 신뢰성 요구사항

ID	항목	요구 수준
NFR-R-01	스레드 안전성	SQLite 다중 스레드 접근 시 mutex 또는 SERIALIZED 모드로 보호
NFR-R-02	예외 처리	HTTP 파서는 비정상 요청(헤더 누락, body 없음)에 크래시 없이 동작
NFR-R-03	DB 자동 초기화	logger_init() 호출 시 DB 파일과 테이블이 없으면 자동 생성

Table 13: 신뢰성 요구사항

4.3 유지보수성 요구사항

ID	항목	요구 수준
NFR-M-01	모듈 분리	C 모듈은 server.c, http_parser.c, logger.c, response.c, main.c로 분리
NFR-M-02	빌드 스크립트	Makefile에 -lsqlite3 -lpthread 링크 명시
NFR-M-03	코드 공유	honeypot.h 변경 시 팀원 간 즉시 공유

Table 14: 유지보수성 요구사항

5 인터페이스 명세 (모듈 간 협업)

5.1 데이터베이스 스키마

```

1 CREATE TABLE IF NOT EXISTS attack_logs (
2   id INTEGER PRIMARY KEY AUTOINCREMENT,
3   timestamp TEXT NOT NULL, -- ISO 8601: "2024-01-01T12
   :00:00Z"
4   ip TEXT NOT NULL, -- IPv4 / IPv6
5   method TEXT NOT NULL, -- "GET", "POST", ...
6   path TEXT NOT NULL,
7   user_agent TEXT,
8   body TEXT,
9   attack_type TEXT -- NULL (C ) → Kotlin UPDATE
10 );

```

Listing 2: attack_logs 테이블 스키마

5.2 포트 및 경로 규약

항목	값	설명
SQLite 파일 경로	/data/honeypot.db	Docker 볼륨으로 공유
timestamp 포맷	2024-01-01T12:00:00Z	ISO 8601, Z 접미사 필수
C 서버 포트	8080	허니팟 수신 포트
Kotlin API 포트	8081	REST API 서비스 포트
React 포트 (dev)	5173	Vite 개발 서버
React 포트 (prod)	3000	Docker 컨테이너

Table 15: 포트 및 경로 규약

5.3 INSERT 규약

C 모듈이 attack_logs에 삽입 시 반드시 아래를 준수한다.

1. timestamp는 strftime 또는 동등한 방법으로 ISO 8601 형식 생성
2. attack_type은 항상 NULL로 삽입 (직접 설정 금지)
3. body가 없는 GET 요청은 빈 문자열("") 또는 NULL로 삽입

6 Docker 배포 요구사항

6.1 컨테이너 구성

서비스	베이스 이미지	포트	비고
core	ubuntu:22.04 등 C 컴파일 환경	8080	-lsqlite3 -lpthread 빌드
api	gradle:8.8-jdk17	8081	installDist로 배포용 스크립트 생성
dashboard	node:20-alpine	3000	Node 18 사용 금지 (CustomEvent 미지원)

Table 16: Docker 컨테이너 구성

6.2 Dockerfile 주의사항

주의

- dashboard: node:18-alpine 사용 불가 → node:20-alpine 필수
- dashboard: npm ci 대신 npm install 사용 (lock file 버전 불일치 방지)
- api: gradle --no-daemon installDist 명령으로 배포용 실행 스크립트 생성
- SQLite DB 파일은 Docker 볼륨으로 core와 api 컨테이너가 공유

7 구현 체크리스트

7.1 원재준 (core/)

완료	파일	요구 사항
☐	honeypot.h	구조체·상수 확정 후 도현서에게 공유
☐	server.c	멀티스레드 소켓 서버 구현
☐	http_parser.c	method, path, User-Agent, body 파싱
☐	logger.c	logger_init() 시 DB·테이블 자동 생성, WAL 모드
☐	logger.c	INSERT 시 attack_type = NULL
☐	logger.c	멀티스레드 SQLite 접근 보호
☐	response.c	경로별 가짜 응답 반환
☐	Makefile	-lsqlite3 -lpthread 링크 추가

Table 17: 원재준 구현 체크리스트

7.2 도현서 (api/ + dashboard/)

완료	파일	요구 사항
✓	Database.kt	Exposed ORM 테이블 정의 + SQLite 연결
✓	AttackClassifier.kt	5초 주기 분류 로직 + UPDATE
✓	Routing.kt	/api/logs, /api/stats, /api/top-ips
✓	Application.kt	플러그인 등록 + DB 초기화 + 분류기 시작
✓	api/index.js	fetchLogs, fetchStats, fetchTopIps
✓	StatCard.jsx	공격 유형별 카드
✓	StatsChart.jsx	파이차트
✓	LogTable.jsx	필터 + 페이지네이션 테이블
✓	TopIps.jsx	바차트
✓	App.jsx	조합 + 10초 자동 갱신
□	—	./gradlew run 로컬 실행 확인 (JDK 17+ 필요)
□	—	curl로 각 엔드포인트 응답 확인
□	—	core/ 완성 후 실제 데이터로 UI 확인

Table 18: 도현서 구현 체크리스트

8 통합 및 테스트 계획

8.1 통합 순서

1. [원재준] core/ 구현 완료 → DB에 실제 데이터 INSERT 확인
2. [도현서] api/에서 SQLite DB 연결 및 분류기 동작 확인
3. [공동] docker-compose up --build로 세 컨테이너 동시 실행
4. [공동] http://localhost:3000에서 대시보드 실제 데이터 확인

8.2 단위 테스트 항목

담당	테스트 항목	검증 방법
원재준	소켓 서버 동작	curl http://localhost:8080/ 요청 후 응답 확인
원재준	HTTP 파서 정확성	다양한 raw HTTP 문자열로 구조체 값 검증
원재준	SQLite INSERT	DB 파일 직접 쿼리(sqlite3 /data/honeypot.db)
원재준	가짜 응답 반환	경로별 curl 요청으로 응답 코드·내용 확인
도현서	API 응답 형식	curl로 각 엔드포인트 JSON 형식 확인
도현서	분류기 동작	NULL 행 삽입 후 5초 대기 → attack_type 업데이트 확인
도현서	대시보드 렌더링	브라우저에서 차트·테이블 정상 표시 확인

Table 19: 단위 테스트 항목

9 변경 이력

버전	날짜	작성자	변경 내용
1.0	2025	도현서	초안 작성

Table 20: 변경 이력